

ROBOQUEST: GENERALIST PHYSICAL AGENTS THAT SEARCH, INSPECT AND TEST

Liu Renhang¹ Navonil Majumder¹ Tej Deep Pala¹ Soujanya Poria¹

¹Nanyang Technological University

Project page: <https://declare-lab.github.io/RoboQuest/>

ABSTRACT

Recent advances in multimodal foundation models have made them capable generalist physical agents for a range of manipulation tasks. However, successful operation in an unfamiliar environment may require an agent to seek task-relevant information through interaction when it is absent from the observations: it may need to determine where a relevant object is, inspect an unobserved property, or discover the effect of an unfamiliar tool. We thus introduce RoboQuest, a benchmark for goal-directed embodied exploration, where agents must actively acquire task-relevant information through physical interaction, use the resulting evidence to adapt subsequent actions, and autonomously decide when to commit to task completion. RoboQuest comprises ten mobile manipulation tasks centered on three forms of uncertainty: search, manipulation-based inspection, and interactive testing. We evaluate five frontier multimodal agents through a common visuomotor interface, as well as a $\pi_{0.5}$ policy fine-tuned on the full-episode demonstrations we release. The best agent succeeds in only 23% of the episodes, and the fine-tuned policy almost never succeeds. Isolated tests of the execution skills the tasks are built from, with the hidden information supplied, show that the agents can carry out most of the required actions, and our failure analysis attributes only a minority of the failures to execution. Our failure analysis further finds that the agents often stop exploring too early as they make decisions before observing the required evidence for task completion. We also find that agents rarely prevent or repair the disturbances caused by their exploration. Moreover, learning by trial and error remains difficult for most models.

1 INTRODUCTION

Recent advances in generalist robotic agents have enabled increasingly complex manipulation, yet such strong execution may not necessarily translate into successful task completion without ready access to key task-relevant information in the observation. In such settings, the robot must actively determine what it needs to know, how to obtain that information, and how each new observation should shape what it does next. This may require searching an environment, inspecting objects from informative configurations, or deliberately interacting with the world to reveal its behavior. Current robotic manipulation benchmarks (Nasiriany et al., 2026; Chen et al., 2026a; Tao et al., 2024; Li et al., 2026b) do not demand such open-ended information-seeking physical interactions from the agents and policies. With the increasing preponderance of generalist robotic agents, such exploratory benchmarking becomes pivotal to truly evaluate the zero-shot capabilities of such agents that may encounter unseen challenges in new environments.

Consider a situation where a robot is searching an unfamiliar room in darkness with only a portable light source. The robot can observe only what the light is aimed at. The challenge here comes from the dual use of the same gripper to carry the light and retrieve objects as well. Exploration is also consequential; for example, a searched compartment may contain the target, nothing, or a key that can be used to access other regions. In other words, each observation can change both its beliefs and next possible actions. The robot must therefore decide where to look, how each discovery should redirect its behavior, and when it has learned enough to act. Similar challenges arise beyond search

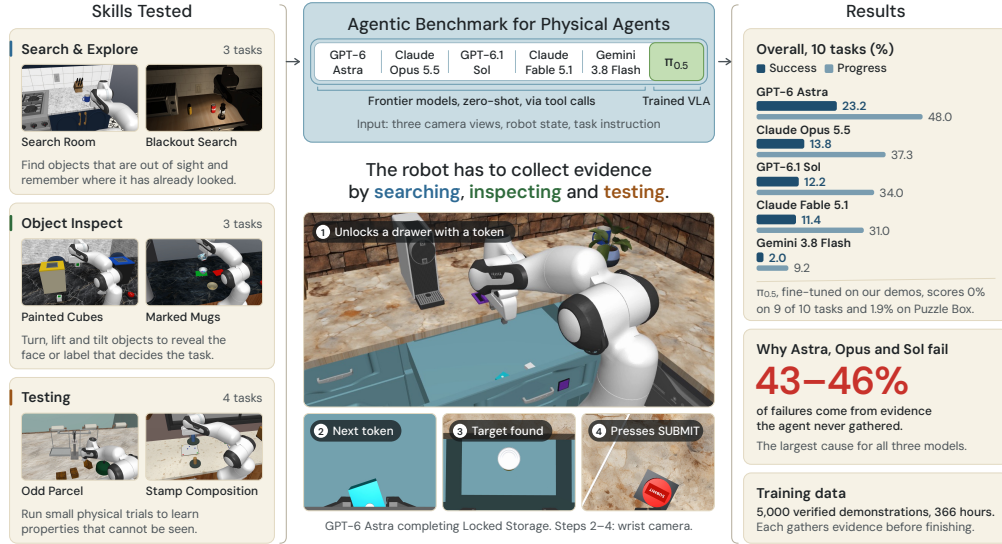


Figure 1: A visual depiction of the salient features of RoboQuest that includes tasks demanding *information-seeking physical interactions* with active perception.

when a robot must determine whether an object has been sufficiently inspected or choose a physical test to probe certain hidden property of an object.

The underlying principle is longstanding. Active and interactive perception study acquiring task-relevant information through decisive actions rather than passively consuming fixed observations (Bajcsy, 1988; Bohg et al., 2017). However, evaluating this capability requires more than simply hiding information from the initial observation. If every uncertainty has an obvious one-step resolution, successful completion primarily tests if the agent performs that prescribed interaction. Thus, the challenge is beyond simply unveiling missing information, that is autonomously conducting an effective investigation in service of a physical goal, where different actions may expose different evidence. Furthermore, information may need to be accumulated across interactions and acquiring them may affect the state of the task. This question is becoming increasingly palpable as frontier multimodal agents such as GPT-6 Astra can now act directly as robot policies across diverse and previously unseen manipulation tasks without task-specific fine-tuning. Improved execution shifts the bottleneck toward deciding what to investigate, how to investigate it, and when enough has been learned.

Thus, we introduce **RoboQuest**, a benchmark for assessing *goal-directed embodied exploration*, in which a robot must autonomously gather task-relevant evidence through physical interaction, use that evidence to guide subsequent actions, and decide when it knows enough to commit to a solution. RoboQuest contains ten tasks organized into three families: **Search & Explore**, **Object Inspect**, and **Testing**. These are mobile manipulation tasks set in full kitchens with a mobile manipulator: exploring often means traversing the kitchen as well as precisely manipulating objects. The tasks involve uncertainty about the location and properties of the relevant objects, and making inferences from the outcomes of physical interactions. Rather than instructing the robot with the investigation approach, each task just specifies an externally verifiable physical goal and leaves the investigation strategy to the agent—from evidence gathering steps to conditions of evidence sufficiency and commitment.

Our evaluations reveal a substantial gap between general manipulation competence and the ability to explore effectively under uncertainty. The best frontier agent succeeds in only 23% of the episodes, and the fine-tuned $\pi_{0.5}$ seldom succeeds. Yet the strongest agents execute most of the required physical actions in isolation when the hidden information is supplied (§4.3), and our failure analysis (§5) attributes only a minority of the failures to execution. The majority involve exploration and decision making. The agents often stop exploring too early, or prematurely make decisions before observing their ramifications. Their exploration also disturbs the scenes that they rarely prevent or repair. Learning by trial and error remains difficult for most models. These results highlight a

persistent gap between stronger robotic execution and the ability to autonomously explore, resolve uncertainty, and achieve a given physical goal.

Our main contributions are:

- **RoboQuest**, a benchmark for goal-directed embodied exploration with ten carefully designed tasks in which exploration shapes subsequent decisions and final task success.
- **A demonstration dataset that explores**, with 5,000 episodes (500 per task, 366 hours at 20 Hz) from scripts that can easily generate more. The demonstrations show how a robot explores and then acts, with a subtask caption on every frame.
- **A systematic evaluation of general-purpose and trained robotic agents**, including five frontier multimodal agents and a RoboQuest-finetuned $\pi_{0.5}$, with isolated tests of the execution skills the tasks require and a detailed failure analysis, which together characterize where exploration under uncertainty breaks down.

2 RELATED WORK

Robotic Manipulation Benchmarks. Standard manipulation benchmarks such as RL-Bench (James et al., 2020), CALVIN (Mees et al., 2022), LIBERO (Liu et al., 2023), ManiSkill3 (Tao et al., 2024), RoboTwin 2.0 (Chen et al., 2025), BEHAVIOR-1K (Li et al., 2023), and RoboCasa (Nasiriany et al., 2024; 2026) evaluate multi-task learning and long-horizon motor control. However, these benchmarks operate under complete or immediately accessible task observability, making the objective primarily execution-centric (Table 1). In contrast, RoboQuest focuses on goal-directed embodied exploration: task-critical information is fundamentally absent from initial observations and cannot be resolved by passive viewing, requiring physical interaction to gather evidence before goal completion.

Comparison with other benchmarks for physical agents. In most manipulation benchmarks, relevant information for task progression and completion is always present in the current observation/state, as compared in Table 1. In RL-Bench, LIBERO, and CALVIN, the agent must exercise the correct motor skills, but need not explore to uncover some necessary information. In contrast, every RoboQuest task requires information acquisition. Moreover, success still depends on perception and motor execution as well. Aggregate success rates therefore cannot isolate and probe a single capability. Hence, we analyze every episode in full to examine what evidence the agent gathered, retained, and used before it committed to some action (§5).

Table 1: **Comparison with representative manipulation benchmarks.** ✓: explicit target; ∼: partial; −: not targeted. *Hidden*: task information absent from the current observation.

Benchmark	Focus	Benchmark features				Evidence seeking		
		Long horizon	Hidden	Memory	Mobile	Search	Inspect	Test
RLBench (James et al., 2020)	Visuomotor skills	∼	∼	−	−	−	−	−
ManiSkill3 (Tao et al., 2024)	Scalable manipulation	∼	−	−	−	−	−	−
RoboTwin 2.0 (Chen et al., 2025)	Bimanual manipulation	∼	−	−	−	−	∼	−
LIBERO (Liu et al., 2023)	Lifelong skill transfer	✓	−	−	−	−	−	−
CALVIN (Mees et al., 2022)	Long-horizon manipulation	✓	∼	−	−	−	−	−
BEHAVIOR-1K (Li et al., 2023)	Household activities	✓	∼	∼	✓	✓	−	−
RoboCasa365 (Nasiriany et al., 2026)	Manipulation in kitchen	✓	−	✓	✓	∼	∼	∼
RMBench (Chen et al., 2026a)	Memory-dependent manipulation	✓	✓	✓	−	−	−	✓
RoboQuest	Evidence-seeking manipulation	✓	✓	✓	✓	✓	✓	✓

Active Perception and Interactive Exploration. When task relevant information is not immediately observable, robots must rely on active perception to acquire missing information (Bohg et al., 2017). While recent benchmarks investigate camera repositioning (Liu et al., 2026b;a), visual occluders (Li et al., 2026b), container retrieval (He et al., 2026; Li et al., 2026a), or history retention (Chen et al., 2026a), they restrict active sensing to viewpoint adjustment or single-step un-occlusion. RoboQuest expands physical exploration into three dimensions: directed search, active inspection, and interactive testing. These exploratory interactions can alter the environment in irreversible ways, forcing agents to reason about the consequences of their actions.

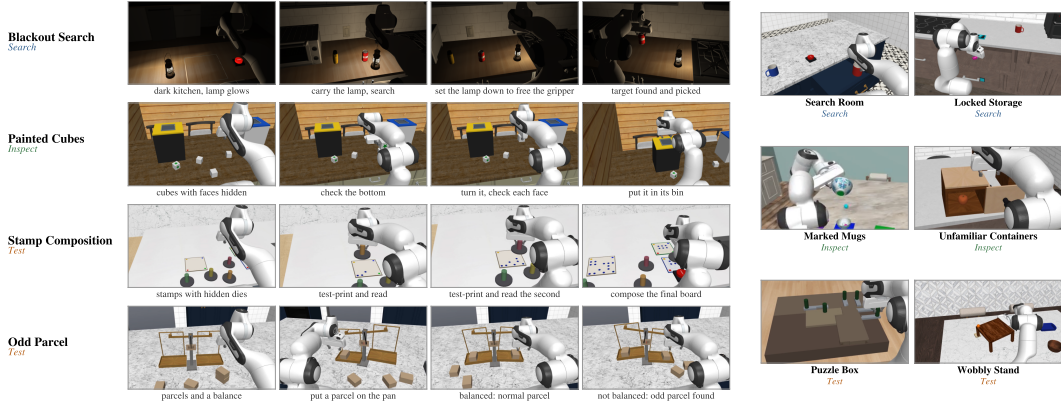


Figure 2: **RoboQuest tasks.** Left: four tasks, from the initial scene through revealing the hidden information to completion. Right: the other six tasks. Colours mark families.

Embodied Reasoning and VLAs under Uncertainty. Robotic foundation models either employ frontier multimodal models as zero-shot high-level planners (Liang et al., 2023; Yang et al., 2025) or train Vision-Language-Action (VLA) models for direct low-level control (Brohan et al., 2023; Kim et al., 2025; Physical Intelligence et al., 2025). Existing evaluations test both paradigms in fully observed scenes. When information must be actively gathered, reasoning agents often prematurely commit or hallucinate unobserved states, while Markovian VLAs struggle with multi-step hypothesis testing. RoboQuest provides an architecture-neutral benchmark to evaluate both paradigms under physical uncertainty.

3 ROBOQUEST BENCHMARK

RoboQuest is a collection of ten tasks—detailed in Appendix D—requiring information-seeking physical interactions for successful completion. As visualized in Fig. 1 the information seeking interactions constitute three dominant types: search, inspect, and test. In *Search* tasks, the missing information is the location of some object. In *Inspect* tasks, it is some hidden information about an object, and the agent reveals it by interacting with the object: turning, lifting, or opening it. In *Test* tasks, the agent cannot observe the hidden information directly. Thus, it must run a series of trials and identify the relevant cue from the outcomes: an impression, a balance, a rolling ball, or a part that moves. We discuss the capabilities required to carry out these tasks in §3.3 and from that lens we also analyze the potential failure modes in §5.

3.1 BENCHMARK SETUP

Tasks. The ten tasks fall into three families by how the missing information is obtained: three Search tasks, three Inspect tasks, and four Test tasks. Figure 2 shows one demonstration of each task, while Table 2 lists what each task hides, and Appendix D describes every task in full, including the parameters its instances vary, with a frame sequence for each in Fig. 6.

Environment. All tasks run in RoboCasa365 kitchens (Nasiriany et al., 2026) simulated in MuJoCo (Todorov et al., 2012), with a Franka Panda arm on a mobile base controlled at 20 Hz and observed by two scene cameras and a wrist camera. Each task places its objects, containers, and mechanisms into a kitchen and states the goal as a natural-language instruction. These are spread over the kitchen, so the robot must drive between them, most of all in the Search tasks. The instruction never gives the hidden information of Table 2, which is kept in a private specification inaccessible to the agent.

Episodes and submission. An episode ends when the robot physically presses a SUBMIT button in the scene. This press freezes the score, so only the state at that moment counts, and the agent itself must judge when its evidence is sufficient. Some actions cannot be undone before that point:

Table 2: **Tasks at a glance.** What each task hides and how the agent can reveal it. Table 9 (Appendix E) lists the capabilities each task targets.

Family	Task	Hidden information	How it is revealed
Search	Locked Storage	target behind locks	find the tokens that unlock each compartment
	Search Room	where the targets are	open storage places until the set is complete
	Blackout Search	targets in the dark	carry a light and search what it illuminates
Inspect	Painted Cubes	marks on unseen faces	turn each cube to see every face
	Marked Mugs	labels under vessels	lift or tilt each vessel to read its underside
	Unfamiliar Containers	how containers open	try each opening mechanism
Test	Puzzle Box	bolt blocking order	try moves and observe which parts move
	Stamp Composition	patterns and rotations	print trial impressions on a test surface
	Wobbly Stand	short legs and gaps	place a ball and watch where it rolls
	Odd Parcel	which parcel is odd	compare groups of parcels on a balance

a cube dropped into a bin stays there, a token shut inside a compartment is locked away, and ink on the final surface is permanent. An episode that ends without a press counts as a failure.

Instances. Each instance is generated from a seed and a setting of the task’s main parameters. The seed fixes the kitchen, the object layout, the materials, and the remaining details. Five tasks also vary occlusion, with every task object in view at the start (*Visible*), some object outside every camera view (*Look*), or some object under a cover (*Uncover*), as detailed in Appendix I. We evaluate on 50 instances per task, which cover every level of the task’s main parameters and together span 18 kitchen layouts and 5 kitchen styles. All agents play the same instances.

Scoring. An episode succeeds if and only if every goal condition holds at the press. For example, every target on the tray or every cube in its correct bin. Since a single wrong unit fails the whole episode, we also measure progress, a per-task partial credit between 0 and 1 defined in Appendix F.

3.2 DEMONSTRATION DATASET

To support the training of learned policies, we also release a dataset of successful demonstrations for all ten tasks. The demonstrations are generated by scripted oracles, one per task. An oracle reads the private specification to execute reliably, but its exploration is scripted as that of an agent without this knowledge, since a demonstration that goes straight to the answer would teach a policy nothing about exploration. The oracles open compartments nearest-first until the targets are seen, lift and tilt each vessel to read its label, turn each cube face by face, print test impressions before stamping the final board, weigh parcels on the balance, and test with the ball again after placing a shim (Table 13). In Unfamiliar Containers the oracle visits the boxes nearest-first by route length, and on each box tries the actions its visible knob suggests in a fixed order (a knob on the lid: lift, twist, slide; a knob on a face: pull, press, slide, turn), including on decoy knobs that open nothing; a box that resists twelve tries is left for later. Most episodes therefore contain several failed tries before a box opens.

Size and selection. The dataset contains 500 episodes for each task, 5,000 in total, amounting to 366 hours of interaction at 20 Hz (Table 13). Episodes last 4.4 minutes on average, about 5,300 control steps, from 1.6 minutes in Puzzle Box to 7.5 minutes in Blackout Search, where the oracle carries a lamp from place to place and searches each one in the dark. The 500 episodes of each task cover its workload levels (§I) in equal numbers and spread over the kitchen layouts and styles. No demonstration shares an instance with the evaluation set. Moreover, the evaluation holds out the kitchen styles, which set the textures and materials of the cabinets, counters, and walls, as well as some task configurations, such as certain target patterns in Stamp Composition and bolt-chain configurations in Puzzle Box. Thus, a policy trained on these demonstrations is always evaluated on unseen scenes and configurations.

Every episode is verified, and every frame carries a two-level language annotation, a stage and the subtask within it, written from the robot’s viewpoint. Appendix H describes the verification, the annotations, and the release format.

3.3 CAPABILITIES NEEDED FOR ROBOQUEST TASKS

Completing a RoboQuest task requires two kinds of actions. An information-seeking action reveals information that informs later actions, such as those in the last column of Table 2. A task action directly drives physical progress toward the goal. The two roles can overlap. For example, opening a container may reveal its mechanism and make retrieval possible. These actions draw on four groups of capabilities: *evidence acquisition* (finding and revealing the missing information), *evidence use* (integrating and remembering what has been observed), *interactive inference* (inferring hidden structure from the outcomes of interactions), and *action organization* (planning action sequences and anticipating their consequences). Appendix E details nine finer capabilities within these groups, and Table 9 marks which of them each task targets.

Our evaluation separates execution from the rest. §4.3 tests the execution skills in isolation, and §5 attributes each failure to one of four families that map onto these groups. Missing evidence mainly reflects weak evidence acquisition, wrong decisions weak evidence use or interactive inference, and side effects weak action organization, while execution failures reflect control, the part that §4.3 measures.

4 EXPERIMENTS

We evaluate RoboQuest in three parts. First, five frontier multimodal models act as zero-shot robot policies through a common visuomotor interface (§4.2); this is the primary focus of the work. Second, to separate exploration from execution, the three models with the highest success rates are tested on the execution skills the tasks are built from, each in isolation and with the information the full task would have to discover supplied up front (§4.3). Third, a $\pi_{0.5}$ policy fine-tuned on our demonstrations serves as a learned-policy baseline. As a memoryless VLA model, it shows how effectively the standard policy-learning approach performs on tasks requiring exploratory interaction (§4.4).

4.1 EXPERIMENTAL SETTINGS

Frontier Agents. Owing to the impressive multimodal capabilities of the current frontier models and their success in robotic control (Zhi, 2025; Anthropic, 2026; Chen et al., 2026b), we evaluate five of them zero-shot: GPT-6 Astra, Claude Opus 5.5 (Opus 5.5 below), GPT-6.1 Sol, and Claude Fable 5.1 (Fable 5.1) with medium thinking, and Gemini 3.8 Flash with high thinking. Every model plays the same 50 evaluation instances of each task (§3.1), 500 episodes per model.

Interface. All models run in the same harness, built on the Inspect Robots agent framework (Robocurve, 2026) and adapted for RoboQuest (Appendix B). At each decision the model receives its proprioception, the elapsed simulated time, the number of decisions remaining, and three 512×512 images from two scene cameras and a wrist camera. It replies with exactly one of four tool calls (Table 6): ARM moves the gripper, BASE drives the mobile base, WAIT holds both, and STOP abandons the episode. The model chooses how long each command runs, up to 10 seconds of simulated time, and there are no tools that locate or grasp objects for it. Besides a SUBMIT press, an episode ends when the model stops or replies without a tool call, or after 200 decisions. The conversation accumulates for the whole episode, including the model’s earlier reasoning, and only the images are pruned, to the two most recent observations. The system prompt and tools are given in Appendix A.

VLA Model Evaluation. To see how far a learned policy gets on these tasks, and because frontier models are often impractical to deploy for reasons of cost and latency, we also evaluate a supervised fine-tuned (SFT) $\pi_{0.5}$ model (Physical Intelligence et al., 2025), trained on the demonstrations of §3.2 and evaluated on the held-out evaluation instances of RoboQuest. The model takes three camera views (left and right base cameras, plus the wrist camera), 16-D robot proprioception, and the task prompt, predicting a subtask label and 20-step action chunks (1.0 s at 20 Hz). Training minimizes a combined objective of flow-matching action loss and causal subtask prediction. Unlike the language-model agents, it acts at the control rate, sending arm, gripper, and base commands at 20 Hz, on the same physics and with the same scoring. At test time, the policy executes closed-loop replanning in

Table 3: **Main results.** SR: success rate. Prog: mean progress. Both in %, 50 episodes per cell. Best SR per row in bold; shading darkens from 0 to 100%. Models are ordered by overall SR. Overall w/o Puzzle Box excludes the easiest task, an outlier on which every model scores far above its other tasks.

Family	Task	GPT-6 Astra		Claude Opus 5.5		GPT-6.1 Sol		Claude Fable 5.1		Gemini 3.8 Flash	
		SR	Prog	SR	Prog	SR	Prog	SR	Prog	SR	Prog
Search	Locked Storage	28.0	52.9	6.0	35.2	12.0	38.5	8.0	21.7	0.0	9.2
	Search Room	0.0	31.0	0.0	26.7	0.0	19.4	0.0	19.2	2.0	17.5
	Blackout Search	0.0	23.7	0.0	17.4	0.0	18.5	0.0	17.1	0.0	4.4
Inspect	Painted Cubes	30.0	74.8	14.0	59.2	18.0	69.2	14.0	61.5	2.0	11.3
	Marked Mugs	38.0	69.7	16.0	51.4	6.0	37.2	12.0	31.8	0.0	0.8
	Unfamiliar Containers	6.0	32.2	0.0	17.2	0.0	23.1	0.0	17.0	0.0	6.3
Test	Puzzle Box	96.0	99.6	68.0	85.5	72.0	90.9	70.0	81.0	12.0	33.0
	Stamp Composition	14.0	55.3	2.0	39.0	8.0	29.4	2.0	36.9	0.0	3.4
	Wobbly Stand	10.0	18.6	24.0	32.2	4.0	7.3	6.0	14.4	2.0	4.6
	Odd Parcel	10.0	22.3	8.0	9.7	2.0	6.7	2.0	9.0	2.0	2.0
Overall		23.2	48.0	13.8	37.3	12.2	34.0	11.4	31.0	2.0	9.2
Overall w/o Puzzle Box		15.1	42.3	7.8	32.0	5.6	27.7	4.9	25.4	0.9	6.6

the simulator until physical `SUBMIT` or task timeout. Training and inference procedures are detailed in §C.

Metrics. *Success rate* (SR, in %) is the share of episodes whose goal state held at a valid `SUBMIT` press. *Progress* (Prog, in %) is the mean of a per-task, staged measure of how much of the goal state was reached, defined in Appendix F; it is computed on the final state of every episode, however it ended, and credits partial results in failed episodes. We also record how each episode ended (correct or wrong submission, stop, text answer, or out of decisions) and, for the frontier models, the tokens, monetary cost at list prices, and wall-clock and simulated time per episode. The isolated skill tests report the success rate of each skill.

4.2 FRONTIER MODELS ON ROBOQUEST

Table 3 and Fig. 3 show a clear lead for GPT-6 Astra, which has the highest success rate (SR) overall (23.2%) and on 7 of the 10 tasks. Opus 5.5, GPT-6.1 Sol, and Fable 5.1 follow close to one another (11–14%), Gemini 3.8 Flash trails at 2.0%, and mean progress follows the same order. The two open search tasks are hard for every model: no model solves a single Blackout Search episode, and the only Search Room success is one episode of Gemini 3.8 Flash. On the opposite end, Puzzle Box is the easiest task for every model, 44 to 58 points above the next best task for the four stronger models, so we also report the averages without it. Without Puzzle Box, GPT-6 Astra succeeds about 1.9 to 3.1 times as often as Opus 5.5, GPT-6.1 Sol, and Fable 5.1. The only task on which another model clearly outperforms GPT-6 Astra is Wobbly Stand, where Opus 5.5 leads in both SR and progress.

Cost Analysis. Evaluating the five models on 500 episodes cost \$21,352 in total, from \$997 for GPT-6.1 Sol to \$9,610 for Fable 5.1. GPT-6.1 Sol is by far the cheapest model, both per episode and per successful episode, and GPT-6 Astra and Opus 5.5 cost about three times as much per success (Fig. 3, Table 7). Fable 5.1 and Gemini 3.8 Flash are the least economical per success, Fable 5.1 because it is the most expensive per episode and Gemini 3.8 Flash because it rarely succeeds. The models make a similar number of decisions but spend very different amounts of time, and the two GPT models command the longest robot motions (Table 7).

How Episodes End. The models give up in different ways (last panel of Fig. 3, Table 10). GPT-6 Astra and GPT-6.1 Sol almost always press `SUBMIT`, but mostly with the task unfinished: only 27% and 16% of their submissions are correct. Opus 5.5 and Fable 5.1 instead often stop on their own, and Opus 5.5 is the most precise model when it does submit (30%). Gemini 3.8 Flash and GPT-6.1

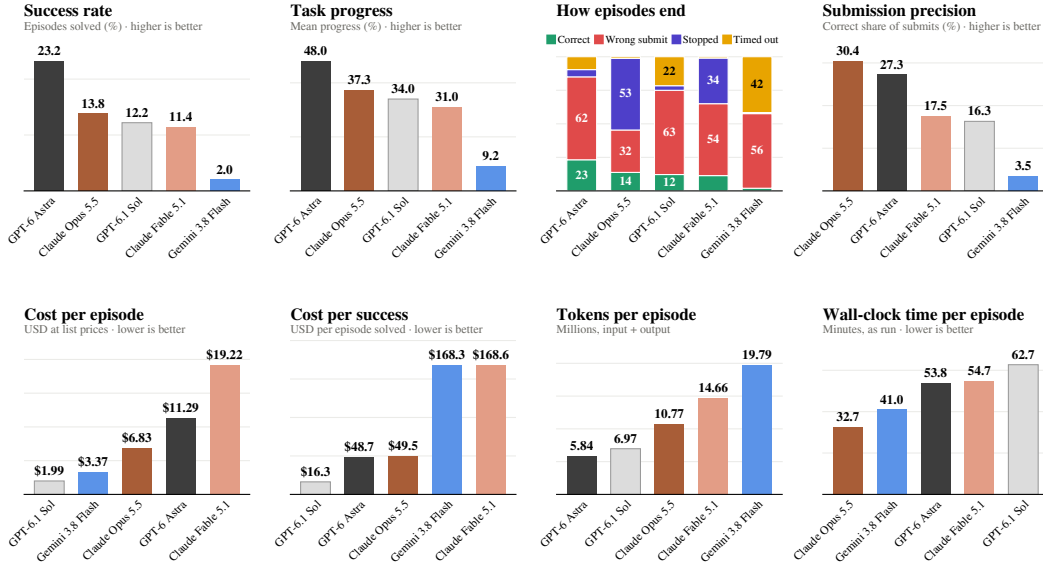


Figure 3: **Results per model at a glance.** Top: success, progress, how the episodes ended, and submission precision, the share of submissions that were correct. Bottom: cost, tokens, and wall-clock time per episode. All values are over the 500 episodes of each model. Bars are ordered from best to worst, except in How episodes end, where “Stopped” counts episodes the model ended on its own, by a stop call or a text answer, and “Timed out” episodes that ran out of the 200 decisions. Exact values: Tables 3, 7 and 10.

Sol run out of the 200 decisions far more often than the others. No model stops because it is done, and no stopped episode had its goal state met. Since the remaining decisions are shown at every turn, a stop and a timeout are two reactions to the same state of little budget and little progress. Opus 5.5 stops with a median of 24 decisions left, where GPT-6 Astra submits what it has and GPT-6.1 Sol runs out. We read the two together as giving up.

4.3 EXECUTION SKILLS IN ISOLATION

Task failures on RoboQuest can come from either ineffective exploration or poor manipulation. To separate the two, we test these execution skills in isolation with the three models achieving the highest success rates, GPT-6 Astra, Opus 5.5, and GPT-6.1 Sol. To isolate execution, we remove the need for exploration by providing information normally uncovered during a task, either in the instruction or by placing a single unambiguous target in the scene. We group the evaluated skills into two categories (Table 4). *General skills* are common across the benchmark, including picking and placing objects on a target and driving the base to an item. *Task-specific skills* are operations required by individual tasks, such as opening a drawer or cabinet, opening an unfamiliar container from instructions, turning a cube, and sliding a shim under a stand. Each skill test starts directly in front of the target, omits the SUBMIT requirement, terminates once the goal is reached or after 50 decisions, and uses the same harness as the main evaluation (Appendix G).

As shown in Table 4, all three models demonstrate strong proficiency on general skills. GPT-6 Astra and Opus 5.5 drive to an object and pick it up in every episode and GPT-6.1 Sol in 90%, while all three place an object on a counter target in 90–95% of the trials. Performance on task-specific skills is more varied. Certain operations achieve consistently high success, such as turning a cube (100%) and opening an unfamiliar container when instructed (90–100%). In contrast, manipulation involving furniture or tight clearances is much harder. Models open a drawer or cabinet in only 30–60% of episodes, retrieve an object from one in 60–70%, and slide the shim under the named table leg in 35–60%. These failures are rarely due to an inability to reach the target. The robot touches the handle in 75–85% of the drawer and cabinet episodes and brings the shim within 3 cm of the

Table 4: **Execution skills in isolation.** Success rate (%), 20 scenes per skill. Best per row in bold. Grey: mean decisions of the successful episodes. Details in Appendix G.

	Skill	Used in	GPT-6 Astra	Claude Opus 5.5	GPT-6.1 Sol
General	Pick and place (on counter)	All tasks	95 (19)	90 (21)	95 (18)
	Drive and pick	Most tasks	100 (23)	100 (23)	90 (23)
Task-specific	Open drawer/cabinet	Search tasks	50 (22)	60 (31)	30 (36)
	Pick from drawer/cabinet	Search tasks	70 (15)	60 (17)	60 (20)
	Open named container	Containers	100 (20)	90 (16)	95 (21)
	Pick from open container	Containers	90 (20)	90 (24)	70 (19)
	Flip a cube	Painted Cubes	100 (12)	100 (19)	100 (11)
	Shim the short leg	Wobbly Stand	35 (33)	60 (32)	40 (40)
All skills			80 (19)	81 (22)	72 (21)

leg in 90–100% of trials, demonstrating that the difficulty lies in the final, precise manipulation. Surprisingly, we found that the same models do better on two of these skills inside the full tasks. For example, GPT-6 Astra opens 90% of the compartments it tries there, against 50% in isolation, and needs 10 decisions on average instead of 22 (second finding in §5.2).

Across all evaluated skills, the three models perform closely (80%, 81%, and 72%). Notably, Opus 5.5 slightly exceeds GPT-6 Astra in isolation, despite GPT-6 Astra succeeding 1.7 times as often on the full benchmark tasks. Performance differences on RoboQuest are therefore not explained by execution alone.

4.4 FINE-TUNED $\pi_{0.5}$

Fig. 4 shows the performance of the fine-tuned $\pi_{0.5}$ models on held-out evaluation instances across RoboQuest tasks. Overall, $\pi_{0.5}$ performs substantially worse compared to frontier multimodal agents. On Puzzle Box, the policy achieves a 2.0% success rate with an average task progress of 43.3%, demonstrating that it can occasionally unbolt early sliders but fails to navigate the complete multi-stage dependency to extract the object. Across the remaining tasks, performance collapses entirely, recording 0.0% success and near-zero progress, with most episodes failing due to timeout.

This failure highlights the challenge of out-of-distribution (OOD) generalisation in goal-directed exploration. The evaluation instances used novel kitchen styles, materials, task mechanisms and target configurations that are not present in the training dataset (§3.2). Thus, the benchmark cannot be solved by simply memorizing state-action mappings or trajectory sequences from training data. Because standard VLA architectures such as $\pi_{0.5}$ lack explicit episodic memory, minor execution errors in OOD scenes quickly compound, leaving the policy stuck in repetitive loops or idling until the decision horizon expires.

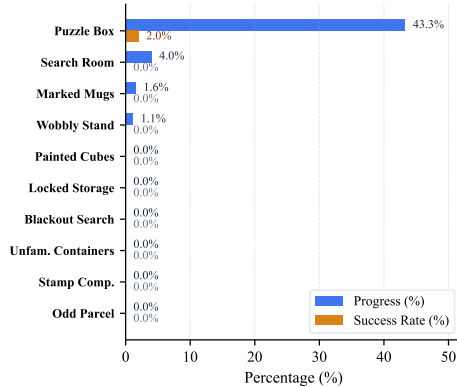


Figure 4: Fine-tuned $\pi_{0.5}$ evaluation results.

5 FAILURE ANALYSIS

§4 showed that frontier models fail in most benchmark episodes, even though they reliably execute the underlying physical skills when hidden information is supplied (§4.3). However, isolated skill tests cannot determine what fraction of failures are caused by execution errors, nor identify what causes the remaining failures. In this section, we attribute each failure of GPT-6 Astra, Opus 5.5, and GPT-6.1 Sol to its initial point of failure: missing evidence, a wrong decision, a disturbed scene,



Figure 5: **Failure attribution by family**, in % of failures, with each failed episode weighted equally, per task and over all tasks. Containers: Unfamiliar Containers. Stamps: Stamp Composition. Counts in Table 16.

or a failed action. Interpreting these results alongside the isolated skill tests reveals that the two analyses are consistent: most failures arise from exploration and decision steps, not from execution of skills.

5.1 ATTRIBUTING FAILURES

To complete a task, an agent repeatedly (i) acquires the evidence it needs, (ii) interprets it and decides what to do (including when to stop), and (iii) executes the decision, observing outcomes and repairing mistakes. We classify failures by the earliest unrecovered breakdown in this loop into four families, two of which arise during physical action:

- **Missing evidence:** the evidence required to fulfill a task goal never reached the agent, not because a physical attempt failed, but because it was never explored. For example, the agent stops or commits to a decision with a hiding place never opened, a cube face never turned to a camera, or a parcel never weighed.
- **Wrong decision:** the necessary evidence appeared in the agent’s observations, but the agent either acted against it (e.g., misreading a label, or stamping the wrong cell after a test print) or failed to act on it (e.g., spotting a target but never retrieving it, or stopping while unfinished work remained in observation).
- **Side effect:** the action itself succeeded but disturbed a task-critical object or state without recovery, e.g., tipping a mug to read its label and spilling balls onto the floor, or accidentally knocking over a stand. Whether the agent failed to foresee the consequence or accidentally collided with the object cannot be determined from the data.
- **Execution failure:** the decided action was appropriate, but its physical execution failed, e.g., slipping off a handle when trying to open a compartment, missing a grasp, dropping an object, placing an item off-target, or failing to recover a dropped item.

Appendix J gives the rules at the boundaries between the families.

Units and procedure. One episode can fail in several ways at once, so we label every *unmet requirement* of a failed episode as a separate unit, not just the first. If a Search Room episode leaves two of three targets off the tray, one never found and one dropped on the way, it gives two units: the first is labelled missing evidence, the second an execution failure. Each unit is labelled at the first unrepaired error on its own path, the sequence of events that led to that requirement’s final state. Each task defines an ordered list of deterministic rules evaluated on every unit. Each rule tests a

verifiable condition, such as whether a compartment was opened, whether an object was visible in the camera observations, or whether a released object landed on the floor. The first rule that matches assigns the failure family to that unit. Appendix J gives the shared measurements and thresholds. The rules of every task and the label of every unit are released with the benchmark. We consider evidence to be observed if it is visible in at least one camera view received by the model. Each failed episode carries equal weight, split over its units, so the episode above counts one half towards each of the two families.

Figure 5 shows the result for 931 / 1143 / 1182 units from 384 / 431 / 439 failed episodes of GPT-6 Astra / Opus 5.5 / GPT-6.1 Sol; values for the three models are given in this order throughout. Missing evidence is the largest family for every model (43 / 43 / 46%), followed by wrong decisions (23 / 31 / 25%); together they account for 66–74% of the failures. Execution failures are 21 / 19 / 18% and side effects 13 / 7 / 12%. The hatched parts are failures in episodes that ran out of decisions (8 / 1 / 19%). For GPT-6 Astra and GPT-6.1 Sol, more than half of these are hiding places that were never opened. §5.2 reads the evidence behind these shares as four failure patterns.

5.2 FAILURE PATTERNS

Incomplete exploration is the main source of failure. Most often, the models stop exploring while parts of the scene remain uninspected. In about half of the failures (45 / 53 / 50%), an object the task needed was never placed (Table 17). In most of these cases (27–29% of all failures), the object remained hidden inside a drawer, cabinet, box, or cover that was never opened. Crucially, the hiding place itself was rarely missed. It typically appeared in the agent’s observations five or more times, while hiding places that were never observed account for only 1–2% of failures. In about a tenth of the failures, the target was fully visible and observed by the agent but never acted on: e.g., a target never fetched, or a dropped object left on the counter.

Beyond stopping exploration early, models also commit to decisions prematurely without gathering sufficient evidence. Roughly a tenth of all failures (9–11%) occurred when models committed objects to locations before observing what was needed to decide correctly. In Odd Parcel, most wrongly selected parcels (71–84%) were boxed before weighings could reveal whether they were odd. In Painted Cubes, marks on uninspected cube faces remained unseen, causing models to misclassify qualifying cubes far more often than unqualified ones (22–37% against 5–10%; Fig. 7). In Stamp Composition, 20–23% of incorrectly placed stamps by GPT-6 Astra and GPT-6.1 Sol were printed without a prior test print (compared to 3% for Opus 5.5).

Controlled experiments varying visual occlusion corroborate this exploration bottleneck (Table 14). When task objects start out of view or under a cover, overall success drops by 8–10 points. Crucially, these additional failures fall entirely under *missing evidence*, while wrong decisions, execution failures, and side effects do not increase (Table 15). In other words, hiding an object makes agents less likely to find it, but no worse at manipulating it once found.

Execution is imperfect but not the bottleneck. Execution failures account for 18–21% of all errors, led by off-target placements and drops to the floor. Surprisingly, opening drawers and cabinets, the skill that fails most in isolation, works better in the full tasks. We analysed the compartments holding a target in Search Room and Locked Storage, and find that models open 75–90% of those they attempt, compared to 30–60% in the isolated test (Table 5, left). At the same time, the successful opening takes fewer decisions, 10 / 26 / 15 on average, against 22 / 31 / 36 in the isolated test (Table 12). Possibly, the robot has already adapted to the controls in the earlier steps of a full episode, while each isolated test starts from scratch. The other two skills that fail most in isolation improve or hold up in the full tasks as well. When trying to grasp a target in an opened compartment, the models take it out in 79–94% of cases, above the isolated 60–70%. In Wobbly Stand, models place a shim under the short leg in 32–60% of the full episodes, where they must identify that leg themselves, against 35–60% in isolation, where the leg is named.

Scene disturbances are rarely prevented or repaired. Side effects account for 7–13% of all errors, representing failures that isolated skill tests cannot capture. In these cases, the model’s intended action succeeds, but inadvertently disrupts a task-critical state. Roughly two-thirds of these errors involve objects falling to the floor where recovery is impossible, such as the test ball in Wobbly Stand rolling off the stand, or balls spilling when a mug in Marked Mugs is tilted to read its

	GPT-6 Astra	Opus 5.5	GPT-6.1 Sol		SR (%)	Prog (%)	Decisions (mean)
<i>Open drawer/cabinet</i>				<i>GPT-6 Astra</i>			
Isolated	50	60	30	No cover	94.4	99.6	84.7
Full task	90	78	75	Covered	96.4	99.5	119.5
Δ	+40	+18	+45	Δ	+2.0	-0.1	+34.8
<i>Take target out</i>				<i>Opus 5.5</i>			
Isolated	70	60	60	No cover	94.4	99.7	94.4
Full task	79	94	81	Covered	36.8	67.4	175.7
Δ	+9	+34	+21	Δ	-57.7	-32.3	+81.2
<i>Shim under a leg</i>				<i>GPT-6.1 Sol</i>			
Isolated	35	60	40	No cover	72.2	90.7	105.8
Full task	58	60	32	Covered	60.1	87.8	150.3
Δ	+23	0	-8	Δ	-12.1	-2.9	+44.5

Table 5: **Left: skills in isolation and in the full tasks.** Success (%) in the isolated tests (§4.3) and in the full tasks (Table 12). In the full tasks, taking the target out counts only targets the model tried to grasp. **Right: Puzzle Box with the locks hidden.** Success rate, progress, and mean decisions per episode, on chain lengths 4 and 5.

label. Emptying the balls is a viable strategy for inspecting the label, and all three models employ it, but they rarely take preventative measures. An empty bowl was provided in each scene for safely holding the balls, but it was used in only 2–8 of the 50 Marked Mugs episodes and never in Wobbly Stand. Furthermore, only 4 of the 38 subsequent attempts to recover a loose ball succeeded.

Interactive trial and error remains challenging. In Puzzle Box, a chain of sliding bolts locks the lid. Without the cover, the blocking dependencies between bolts are directly visible. On the other hand, with the cover, the unlocking sequence must be learned by trying bolts and observing what moves. On matching chain lengths (Table 5, right), adding the cover barely affects GPT-6 Astra, which succeeds in 94.4% of episodes without it and 96.4% with it (using 120 vs. 85 decisions). In contrast, Opus 5.5 drops sharply from 94.4% to 36.8%, while GPT-6.1 Sol falls moderately from 72.2% to 60.1%. Roughly half of Opus 5.5’s failed attempts involve pulling a blocked bolt three or more times without updating its plan (12 of 26). The Unfamiliar Containers task requires a similar capability. Given instructions on the opening mechanism, Opus 5.5 succeeds in 90% of isolated trials (Table 4), but when left to discover the mechanism interactively, it opens only 45% of attempted containers in the full task, against 70% and 78% for GPT-6 Astra and GPT-6.1 Sol.

6 CONCLUSION

The importance of RoboQuest is highlighted by the empirical performance and cost of five frontier models. All five generalist frontier physical agents succeed on only a small fraction of episodes: GPT-6 Astra achieves the highest success rate at 23.2%, Opus 5.5, GPT-6.1 Sol, and Fable 5.1 follow at 11 to 14%, and Gemini 3.8 Flash achieves the lowest at 2%; GPT-6.1 Sol is the most economical per successful episode. Beyond aggregate performance, two complementary analyses show where goal-directed exploration breaks down. When tested in isolation, models succeed on execution skills in 72–81% of attempts, while our failure analysis attributes only 18–21% of failures in the full tasks to execution. The two analyses agree that the bottleneck lies in exploring and decision-making rather than in execution, with most failures resulting from exploration that stops too early. The models stop exploration while parts of the scene remain uninspected, or commit to decisions before acquiring necessary evidence (9–11% of failures). Our controlled occlusion experiments confirm that hiding objects increases missing-evidence errors. Furthermore, physical exploration can have irreversible consequences, disrupting task-critical state in 7–13% of failures, while interactive hypothesis testing remains brittle: covering the locks in Puzzle Box leaves GPT-6 Astra unaffected but cuts Opus 5.5’s success from 94% to 37%. Finally, models diverge in how they end tasks, either submitting prematurely or exhausting their decision budget. Together, these results demonstrate that progress on embodied exploration requires not only stronger perception

and control, but also systematic evidence acquisition, persistent goal and evidence tracking, causal hypothesis testing, and robust monitoring and recovery during physical interaction.

AI USE STATEMENT

In this work, we used generative AI tools to provide feedback on research methodology and experiments, assist with method implementation, create or modify scientific figures, summarise existing literature, and draft or edit portions of the manuscript for clarity. We did not use these tools to develop theoretical models, formulate or prove mathematical claims, propose or refine hypotheses, clean or reformat datasets, generate synthetic data, or translate the manuscript. We manually checked LLM-generated code for correctness, verified literature summaries against the cited sources, and reviewed all AI-generated text and figures. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI.

REFERENCES

- Anthropic. Claude plays robotics. <https://www.anthropic.com/research/claude-plays-robotics>, 2026. Accessed: 2026-09-26.
- R. Bajcsy. Active perception. *Proceedings of the IEEE*, 76(8):966–1005, 1988. doi: 10.1109/5.5968.
- Jeannette Bohg, Karol Hausman, Bharath Sankaran, Oliver Brock, Danica Kragic, Stefan Schaal, and Gaurav S Sukhatme. Interactive perception: Leveraging action in perception and perception in action. *IEEE Transactions on Robotics*, 33(6):1273–1291, 2017.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- Tianxing Chen, Zanzin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xi-anliang Lin, Yiheng Ge, Zhenyu Gu, et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.
- Tianxing Chen, Yuran Wang, Mingleyang Li, Yan Qin, Hao Shi, Zixuan Li, Yifan Hu, Yingsheng Zhang, Kaixuan Wang, Yue Chen, et al. Rmbench: Memory-dependent robotic manipulation benchmark with insights into policy design. *arXiv preprint arXiv:2603.01229*, 2026a.
- Xuchuan Chen, Xiaoqian Cheng, Yu Deng, Lihe Ding, Shaocong Dong, Xiangjun Gao, Haozhe Jia, Zekai Li, Zhoujian Li, Yunrui Lian, Sikai Liang, Chenghuai Lin, Dairu Liu, Jiahang Liu, Qingtao Liu, Yuxuan Ma, Zekun Qi, Jiayi Su, He Wang, Ruochen Xu, Tianyu Xu, Xudong Xu, Zhe Xu, Mi Yan, Siming Yan, Li Yi, Ruixi Yu, Jinlu Zhang, Yintianrun Zhang, Zhikai Zhang, Zhizheng Zhang, Yixin Zheng, and Weiyi Zhu. Systematically exploring the capabilities of GPT-6 Astra as embodied policies. *arXiv preprint arXiv:2609.38537*, 2026b.
- Yuxin He, Ruihao Zhang, Tianao Shen, Cheng Liu, and Qiang Nie. Towards exploratory and focused manipulation with bimanual active perception: A new problem, benchmark and strategy. *arXiv preprint arXiv:2602.01939*, 2026.
- Stephen James, Zicong Ma, David Rovick Arrojo, and Andrew J Davison. Rlbench: The robot learning benchmark & learning environment. *IEEE Robotics and Automation Letters*, 5(2):3019–3026, 2020.
- Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabrael Levine, Michael Lingelbach, Jiankai Sun, et al. Behavior-1k: A benchmark for embodied ai with 1,000 everyday activities and realistic simulation. In *Conference on Robot Learning*, pp. 80–93. PMLR, 2023.

- Jialiang Li, Yi Qiao, Yunhan Guo, Changwen Chen, and Wenzhao Lian. Act, sense, act: Learning non-markovian active perception strategies from large-scale egocentric human data. *arXiv preprint arXiv:2602.04600*, 2026a.
- Taishan Li, Jiwen Zhang, Siyuan Wang, Xuanjing Huang, and Zhongyu Wei. Libero-occ: Evaluating and improving vision-language-action models under scene-induced occlusion via viewpoint imagination. *arXiv preprint arXiv:2606.10862*, 2026b.
- Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International conference on robotics and automation (ICRA)*, pp. 9493–9500. IEEE, 2023.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- Mengzhen Liu, Enshen Zhou, Cheng Chi, Yi Han, Shanyu Rong, Liming Chen, Pengwei Wang, Zhongyuan Wang, and Shanghang Zhang. Sapave: Towards active perception and manipulation in vision-language-action models for robotics. *arXiv preprint arXiv:2603.12193*, 2026a.
- Zhenyang Liu, Yongchong Gu, Yikai Wang, Xiangyang Xue, and Yanwei Fu. Activevla: Injecting active perception into vision-language-action models for precise 3d robotic manipulation. *arXiv preprint arXiv:2601.08325*, 2026b.
- Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters*, 7(3):7327–7334, 2022.
- Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. *arXiv preprint arXiv:2406.02523*, 2024.
- Soroush Nasiriany, Sepehr Nasiriany, Abhiram Maddukuri, and Yuke Zhu. Robocasa365: A large-scale simulation framework for training and benchmarking generalist robots. In *International Conference on Learning Representations (ICLR)*, 2026.
- Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- Robocurve. Inspect robots: The open-source evaluation framework for physical ai, 2026. URL <https://github.com/robocurve/inspect-robots>.
- Stone Tao, Fanbo Xiang, Arth Shukla, Yuzhe Qin, Xander Hinrichsen, Xiaodi Yuan, Chen Bao, Xinsong Lin, Yulin Liu, Tse-kai Chan, et al. Maniskill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai. *arXiv preprint arXiv:2410.00425*, 2024.
- Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033. IEEE, 2012. doi: 10.1109/IROS.2012.6386109.
- Rui Yang, Hanyang Chen, Junyu Zhang, Mark Zhao, Cheng Qian, Kangrui Wang, Qineng Wang, Teja Venkat Koripella, Marziyeh Movahedi, Manling Li, et al. Embodiedbench: Comprehensive benchmarking multi-modal large language models for vision-driven embodied agents. *arXiv preprint arXiv:2502.09560*, 2025.
- et al. Zhi. Closed-loop open-vocabulary mobile manipulation with gpt-4v. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025.

APPENDIX CONTENTS

A System Prompt and Tools	15
B Evaluation Harness	16
C $\pi_{0.5}$ Fine-Tuning and Evaluation Details	16
D Benchmark Tasks	17
E Fine-Grained Capabilities for RoboQuest	20
F Task Progress	20
G Execution Skill Test Details	22
H Demonstration Dataset Details	23
I Workload and Occlusion Details	24
J Failure Attribution Details	24

A SYSTEM PROMPT AND TOOLS

Every model receives the same system prompt at the start of the episode, and only the goal text differs between tasks. The prompt for Puzzle Box reads:

You control a simulated mobile manipulation robot. Use only the provided camera images, robot proprioception and action/budget feedback. Determine what the goal requires from the scene. Choose exactly one arm, base, wait or stop tool per response. There are no semantic object-location or grasp tools. Images are upright RGB from three fixed cameras; new views require physical robot or object motion. The arm tool controls the right gripper site in world metres, with optional world orientation as quaternion xyzw. World z is up. Gripper -1 opens, +1 closes and 0 keeps its current setpoint. The base tool takes normalized body-frame forward, left and yaw velocity commands; positive yaw turns left. The arm stays relative to the base during base motion. The torso is held. A tick is 0.05 seconds. Commands consume up to 200 ticks each, ending early on the first physical `SUBMIT` button press. Check observed outcomes: commanded targets are not guaranteed to be reached. Physically press `SUBMIT` to commit the result; the first press freezes the score and ends the episode immediately. Wrong submission, timeout without submission, or calling stop without submission fails. The stop tool abandons the episode without advancing time. Invalid or multiple tool calls perform no physics and still consume this model response. No intermediate success is awarded. Goal: Open the wooden puzzle box on the counter, take out the tangerine inside, and stand it inside the tray. The lid slides sideways but is locked by sliding bolts with round black handles: a part moves only when nothing blocks it, so work out the order. Any order that works is fine. Once the tangerine is inside the tray and you have let go of everything, press the red `SUBMIT` button. Your first submission ends the episode; a wrong submission or no submission fails.

Table 6 lists the four tools provided to the agents. After each tool call, the agent receives a response indicating whether the command was accepted, the current simulation tick count, and any error message. To prevent any information leakage, this response never exposes simulator state, runtime exceptions, or task progress.

Table 6: **Action tools.** Positions are in world metres, and a tick is 0.05 s of simulated time. Every tool except `STOP` accepts a tick count from 1 to 200.

Tool	Arguments	Default ticks
ARM	gripper-site position (required), orientation as a quaternion, and gripper command in $[-1, 1]$, where -1 opens and $+1$ closes	120
BASE	body-frame forward, left, and yaw velocity, each in $[-0.5, 0.5]$	20
WAIT	optional gripper command, holds the arm and base	20
STOP	none, abandons the episode, which fails	—

B EVALUATION HARNESS

Overview. The evaluation harness extends the Inspect Robots agent framework (Robocurve, 2026), which provides the model client interfaces and the turn-based execution loop. All remaining components, such as the simulation environments and task scoring, are specific to RoboQuest.

Observations. At each decision step, the model receives observations as a user message. The robot’s proprioceptive state and the remaining decision budget are provided as text input, along with three 512×512 RGB images captured from the left and right scene cameras and the wrist-mounted camera.

Actions and execution. At each step, the model must reply with exactly one tool call and specify the action’s duration in simulation ticks. An `ARM` command servos the gripper toward the target pose using the robot’s operational-space controller. A `BASE` command applies body-frame velocities at each tick while holding the arm fixed relative to the base, and `WAIT` holds both. Each command runs for exactly its requested ticks, with no additional settling time. Commands terminate early only if the physical `SUBMIT` button is pressed.

Memory. Each episode begins with a fresh conversation context, with no memory retained across episodes. Within an episode, each turn appends the observation, the model’s full response (including its reasoning), and the tool execution result to the dialogue history. While the complete textual history is preserved, the image history is pruned, retaining only the two most recent observations consisting of six images in total.

Budgets and termination. Each episode allows a budget of 200 model decisions, with individual commands running up to 200 simulation ticks. An episode terminates upon pressing the `SUBMIT` button, exhausting the decision budget, calling `STOP`, or returning an invalid response. Tasks are scored strictly on the simulator state at the time of episode terminations.

Table 7: **Cost and effort per episode.** Sim / Wall: simulated / wall-clock minutes. In / Out: tokens in millions. Costs in USD at list prices. Best per column in bold (the lowest value, or the highest cache rate).

Model	Effort per episode			Tokens per episode			Cost (USD)	
	Decisions	Sim min.	Wall min.	In (M)	Out (M)	Cached (%)	\$/episode	\$/success
GPT-6 Astra	143.7	10.4	53.8	5.82	0.02	91.8	11.29	48.7
Claude Opus 5.5	143.0	5.4	32.7	10.70	0.07	93.5	6.83	49.5
GPT-6.1 Sol	157.4	11.2	62.7	6.94	0.02	92.4	1.99	16.3
Claude Fable 5.1	148.8	6.1	54.7	14.55	0.11	94.2	19.22	168.6
Gemini 3.8 Flash	160.8	5.1	41.0	19.66	0.13	89.3	3.37	168.3

C $\pi_{0.5}$ FINE-TUNING AND EVALUATION DETAILS

We fine-tune $\pi_{0.5}$ on each task separately, using that task’s demonstrations from our dataset (§3.2).

Observations. At each decision step, the model receives three RGB camera views resized to 224×224 pixels, the robot’s proprioceptive state, and the task instruction.

Action Space and Horizon. The policy predicts action chunks of horizon $H = 20$, corresponding to 1.0 second of continuous execution at the simulator’s 20 Hz control frequency. Each predicted action is a 12-dimensional vector consisting of a 6-D end-effector delta pose, a 1-D continuous gripper action (± 1), 3-D mobile base velocities, and a 2-D torso pose.

Training Objective. Our training simultaneously optimizes an autoregressive language modelling objective for subtask prediction and a conditional flow-matching objective for low-level action chunking:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{subtask}} + 10 \cdot \mathcal{L}_{\text{flow}}, \quad (1)$$

where $\mathcal{L}_{\text{subtask}}$ is the cross-entropy loss over causal subtask tokens, and $\mathcal{L}_{\text{flow}}$ is the flow-matching loss conditioned on vision, state, prompt, and the teacher-forced subtask.

Hyperparameters. Table 8 lists the training and inference hyperparameters used. We perform full-parameter fine-tuning with the AdamW optimizer, cosine learning rate scheduling, and exponential moving average (EMA) weight tracking.

Table 8: $\pi_{0.5}$ **Training and Inference Hyperparameters.**

Category	Hyperparameter	Value
Training	Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$)
	Weight Decay	1×10^{-4}
	Peak Learning Rate	2.5×10^{-5}
	Learning Rate Schedule	Cosine decay with 1000 warmup steps
	Global Batch Size	32
	Gradient Clipping Norm	1.0
	Flow Loss Multiplier	10.0
	Action Normalization	Quantile (q_{01}, q_{99})
Inference	Replanning Frequency	Once per 20 ticks (1.0 s)
	Flow Denoising Steps	10

During evaluation, the policy runs in closed loop, replanning every 20 ticks (1.0 s) until the episode terminates upon pressing the physical `SUBMIT` button or exceeding the task horizon.

D BENCHMARK TASKS

RoboQuest organizes evaluation tasks into three families according to the dominant information-gathering behavior required during execution: *Search & Explore*, *Object Inspect*, and *Testing*.

D.1 SEARCH & EXPLORE

Search Room. A set of target items is stored in open or closed locations around a room, among distractor items of similar kind. The goal is to collect the complete set on the tray and press `SUBMIT`. The locations, the number of storage places, and which storage places are empty are not given. The robot must choose where to search, use negative findings to redirect, remember what it has checked, and stop once the set is complete.

Main parameters: 4, 6, or 8 compartments and 2 or 3 targets.

Goal text: “Collect the $\langle \text{targets} \rangle$ on the tray, then press `Submit`. They may be anywhere in this kitchen; only compartments with handles can be opened. Other objects are not part of the task; you may move them if needed.”

Locked Storage. Storage compartments open only when a matching access token rests on their reader. Tokens are either outside the initial observation or hidden inside other storage, and some

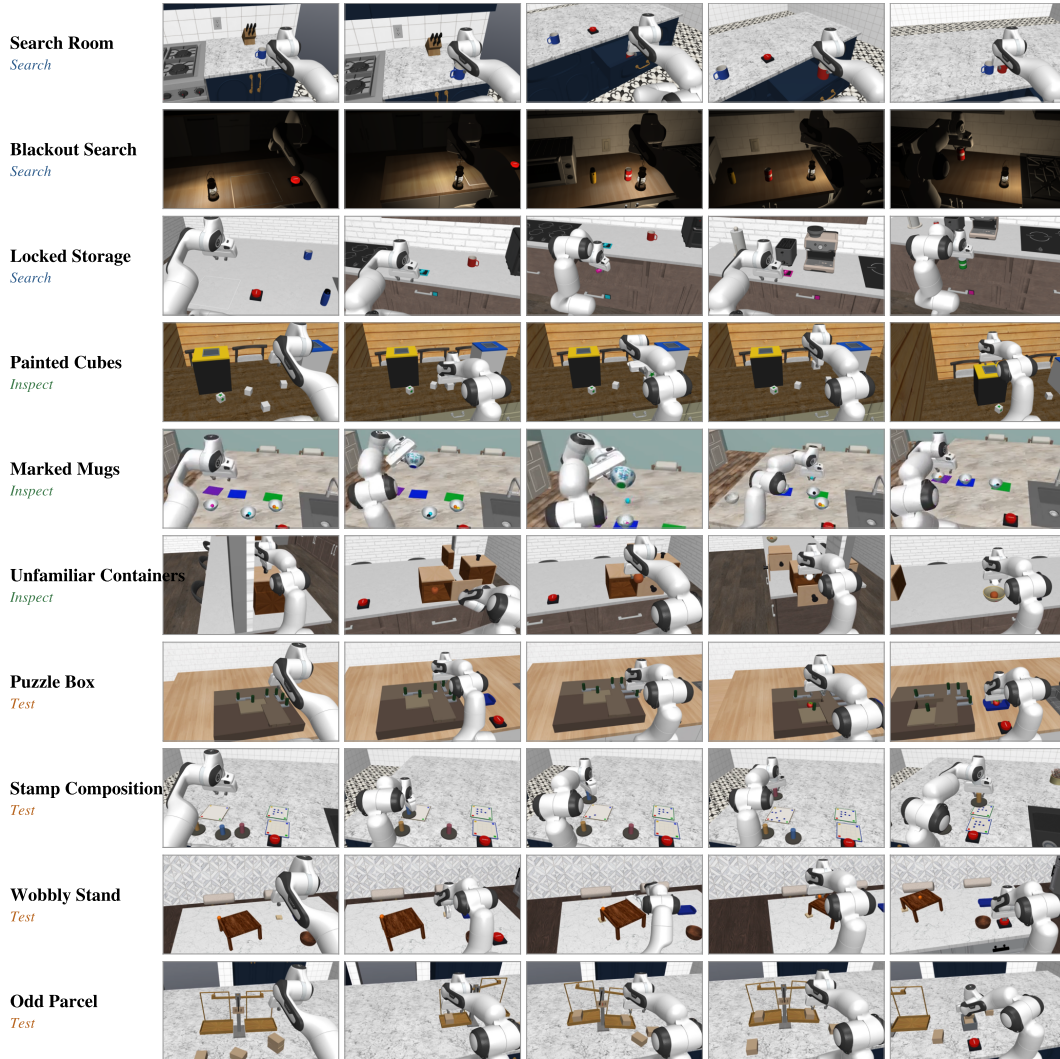


Figure 6: **All ten RoboQuest tasks.** Five frames from one scripted demonstration of each task, from a scene camera, running from the initial scene through the actions that reveal the hidden information to the completed task. Family labels are given in colour.

compartments contain tokens for other compartments, forming a dependency chain. The goal is to retrieve the target item. Finding a token changes which places can be searched next. Closing a compartment with a token still inside locks that token away permanently, so the robot must plan token handling as well as search order.

Main parameters: chain depth 1, 2, or 3.

Goal text: “Collect the $\langle target \rangle$ on the tray, then press `Submit`. It may be anywhere in this kitchen; some compartments may not open, and a compartment with a coloured lock plate opens only while the token of that colour rests on its reader pad. Other objects on the counter are not part of the task; you may move them if needed.”

Blackout Search. A search task is performed in an unlit room. Only a light source with a glow marker and the `SUBMIT` button are visible at the start. The robot must acquire the light source, carry it, and search using a wrist camera that sees only what is illuminated. Because one gripper holds the light, retrieving a target requires putting the light down while aiming it at the target, then acting from memory in the dark.

Main parameters: 4, 6, or 8 compartments and 2 or 3 targets.

Goal text: “The kitchen is dark. Collect the $\langle targets \rangle$ on the tray, then press `Submit`. They may be anywhere in this kitchen; some compartments may not open. A lantern with a glowing glass stands on a counter. Other objects on the counter are not part of the task; you may move them if needed.”

D.2 OBJECT INSPECT

Marked Mugs. Several identical vessels, all mugs or all bowls in a given instance, each hold loose contents, and a label on the underside of each vessel specifies its destination. The goal is to place every vessel upright at its destination with its original contents inside. Reading a label requires lifting or tilting the vessel, which spills the contents. Spilled contents stay nearby and must be returned to the correct vessel before submission.

Main parameters: 3, 4, or 5 vessels and occlusion Visible, Look, or Uncover.

Goal text: “There are $\langle mugs \text{ or } bowls \rangle$ on the counters in this kitchen. Each one has a coloured label on its bottom. Put every one upright on the pad of the same colour, with its own two balls inside, then press `Submit`. Other objects on the counter are not part of the task; you may move them if needed.”

Painted Cubes. A collection of similar objects each carries a hidden property distributed over its faces, such as the number of painted faces, with some faces turned toward the table or away from every camera. The goal is to collect every object satisfying a stated property, for example, exactly one painted face. Partial views can reject an object but cannot confirm it, so the robot must decide how much inspection each object needs.

Main parameters: 4, 6, or 8 cubes and occlusion Visible, Look, or Uncover.

Goal text: “There are $\langle cubes \rangle$ on the counters in this kitchen. Some of them satisfy this rule: every cube $\langle rule \rangle$. Put every cube that satisfies it in the blue bin and every other cube in the yellow bin, then press `Submit`. Other objects on the counter are not part of the task; you may move them if needed.”

Unfamiliar Containers. Several closed containers each open by a different mechanism, such as a hinged lid, sliding lid, drawer, or latched flip-top, and some cannot be opened at all. The target is inside one of them. The robot knows neither where the target is nor how each container opens and must discover each mechanism by interaction.

Main parameters: 2, 3, or 4 items and occlusion Visible or Look.

Goal text: “There are $\langle boxes \rangle$ on the counters in this kitchen. $\langle Items \rangle$ are inside them. Put $\langle all \text{ of } them \rangle$ in the bowl, then press `Submit`.”

D.3 TESTING

Stamp Composition. Several stamps with unmarked housings each carry a hidden pattern and start at an unknown rotation. A target pattern is shown on a reference. A test surface allows trial impressions, and a final surface must end with exactly the target. The robot must test stamps, read the results, select a subset and their rotations, compose the target, and press `SUBMIT`. Marks on the final surface are permanent.

Main parameters: 2, 3, or 4 stamps and occlusion Visible, Look, or Uncover.

Goal text: “There are $\langle stamps \rangle$ on the counters in this kitchen. Stamp the final board so it shows exactly the pattern on the reference card, then press `Submit`. The final board is the blue-bordered one with the printed 3x3 grid; the green-bordered reference card is not for stamping. The round self-inking stamps stand face down on the work surface. Each prints a fixed pattern of dots in whatever direction it is held, and the unmarked housings show neither the pattern nor which way it points, so try them on the tan scratch board. Ink stays where it lands, on the scratch board and on the final board alike. Fill exactly the reference card’s grid cells with no ink outside the final board’s grid, put every stamp back down on the counter and let go, then press the red `SUBMIT` button. Your

first submission ends the episode; a wrong submission or no submission fails. Other objects on the counter are not part of the task; you may move them if needed.”

Odd Parcel. A set of visually identical sealed parcels contains one or two parcels that differ in weight. The only instrument is a two-pan balance whose pans hold several parcels each. The goal is to place the odd parcel alone on the tray. Group weighing identifies it in few comparisons, while one-by-one weighing is legal but slower.

Main parameters: 4, 6, or 8 parcels and occlusion Visible, Look, or Uncover.

Goal text: “There are $\langle parcels \rangle$ on the counters in this kitchen. Most of these parcels weigh the same; fewer than half weigh differently. Put every parcel that weighs differently in the box and nothing else, leave the balance where it stands, then press `Submit`. Other objects on the counter are not part of the task; you may move them if needed.”

Puzzle Box. A container is held closed by a chain of interlocking parts, where each part blocks another until it is moved. All parts are visible with large handles, but the blocking relations are not. The goal is to place the item inside on the tray. The robot must discover the release order by trying moves and observing what moves, and some wrong moves jam another part until reversed.

Main parameters: chain length 4, 5, or 6 and cover absent or present.

Goal text: “Open the wooden puzzle box on the counter, take out the $\langle item \rangle$ inside, and stand it inside the tray. The lid slides sideways but is locked by sliding bolts with round black handles: a part moves only when nothing blocks it, so work out the order. Any order that works is fine. Once the $\langle item \rangle$ is inside the tray and you have let go of everything, press the red `SUBMIT` button. Your first submission ends the episode; a wrong submission or no submission fails.”

Wobbly Stand. A stand or table has an uneven support, tilting its surface by an amount too small to see directly. A ball placed on the surface rolls toward the low side and reveals the tilt. Shim blocks of different thicknesses are available. The goal is a level surface on which the ball stays put, with everything released.

Main parameters: 1 or 2 short legs and shim choice absent or present.

Goal text: “The stand is not level. Use the shims to level it so the ball stays still on top, then press `Submit`.”

E FINE-GRAINED CAPABILITIES FOR ROBOQUEST

Table 9 lists the capabilities each task targets. Directed search is the core of the Search family, active perception of the Inspect family, and experimental identification of the Test family. Evidence integration and memory are needed wherever an observation only matters together with earlier ones. Affordance discovery is targeted in Unfamiliar Containers, where the hidden property is how each box opens. Physical causal inference is targeted in Puzzle Box and Wobbly Stand, where the agent learns the hidden structure from how the scene responds to its actions.

Consequential action is marked where gathering information can damage the goal, for example spilling a mug’s contents or locking a token away. Long-horizon planning is marked where the steps must be ordered in advance, as in Locked Storage and Puzzle Box. Every task ends with a `SUBMIT` press, so deciding when the evidence is enough to commit is common to all tasks. It is not counted as consequential action in the table.

F TASK PROGRESS

This appendix defines the progress $P \in [0, 1]$ of every task. P is computed on the scene frozen at the first `SUBMIT` press, a stop, or the end of the budget.

Search Room. Each target earns a third of its credit once its compartment is open, two thirds once it has been taken out, and full credit once it stands upright on the tray, released. A target under a

Table 9: **Capabilities each task targets**, with what each task hides ($\checkmark$: directly targeted, $\sim$: supporting, $-$: none). DS: directed search. AP: active perception. EI: evidence integration. M: memory. AD: affordance discovery. XI: experimental identification. CI: physical causal inference. CA: consequential action, marked where an information-gathering step can irreversibly damage the goal. LP: long-horizon planning. The table lists what each task demands, not where agents fail.

Family	Task	Hidden information	Evidence acquisition		Evidence use		Interactive inference			Action organization	
			DS	AP	EI	M	AD	XI	CI	CA	LP
Search	Locked Storage	target behind locks	$\checkmark$	$-$	$\checkmark$	$\sim$	$-$	$-$	$\sim$	$\checkmark$	$\checkmark$
	Search Room	where the targets are	$\checkmark$	$\sim$	$\checkmark$	$\checkmark$	$-$	$-$	$-$	$-$	$\sim$
	Blackout Search	targets in the dark	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$-$	$-$	$-$	$-$	$\sim$
Inspect	Painted Cubes	marks on unseen faces	$-$	$\checkmark$	$\checkmark$	$\sim$	$-$	$-$	$-$	$\sim$	$\sim$
	Marked Mugs	labels under vessels	$-$	$\checkmark$	$\sim$	$\sim$	$-$	$-$	$-$	$\checkmark$	$\sim$
	Unfamiliar Containers	how containers open	$\sim$	$-$	$-$	$\sim$	$\checkmark$	$\sim$	$-$	$-$	$\sim$
Test	Puzzle Box	bolt blocking order	$-$	$-$	$-$	$\sim$	$\sim$	$\sim$	$\checkmark$	$\sim$	$\checkmark$
	Stamp Composition	patterns and rotations	$-$	$-$	$\checkmark$	$\sim$	$-$	$\checkmark$	$-$	$\checkmark$	$\checkmark$
	Wobbly Stand	short legs and gaps	$-$	$-$	$\sim$	$\sim$	$-$	$\checkmark$	$\checkmark$	$\checkmark$	$\sim$
	Odd Parcel	which parcel is odd	$-$	$-$	$\checkmark$	$\checkmark$	$-$	$\checkmark$	$-$	$-$	$\sim$

cloche or on the open counter earns nothing for opening. Credit for opening and taking out is kept even if the agent later closes the compartment or drops the target. P is the mean credit over the targets, less 0.1 if a non-target object is on the tray. A compartment counts as open at 15% of its joint range, which is 0.15 m for a drawer and 0.24 rad for a door. A target counts as taken out once it leaves its compartment or cloche, or once it moves 0.15 m from an open spot.

Locked Storage. The locks of the chain and the target share P equally. Each lock that is open at the end earns its share, and the target earns its share as in Search Room. Once the target has been taken out, every lock counts as open. The dead-end compartment is not part of the chain. P is reduced by 0.1 if a non-target object is on the tray.

Blackout Search. Same as Search Room.

Marked Mugs. $P = \frac{1}{n} \sum_i c_i b_i$. Let f_i be the fraction of the footprint disc of vessel i (radius 3.95 cm) that lies on the pad of its label colour. The placement credit c_i is 1 if $f_i \geq 3/4$, 0.5 if $1/2 \leq f_i < 3/4$, and 0 otherwise, and it counts only while the vessel is upright, released, still and supported by that pad. A vessel on a wrong pad scores 0. $b_i = 1$ if the vessel holds its own two balls, else 0.5. Success requires $c_i = b_i = 1$ for every vessel.

Painted Cubes. $P = c/n$, where c is the number of cubes captured in their correct bin. Captures are irreversible.

Unfamiliar Containers. As Search Room, over the items. An item earns a third of its credit once its box has been opened, two thirds once it is out of the box, and full credit once it is in the bowl.

Stamp Composition. $P = \sum_{c \in C} g_c / (|C| + m)$, where C is the set of target cells. A cell scores $g_c = 1$ if it holds only clean dots, each centred within 9 mm, and at most its k_c dots. It scores $g_c = 0.5$ if a dot crosses the cell edge or it holds more than k_c dots, and $g_c = 0$ if it is empty. m counts the stray dots outside C , plus one if the ink outside the grid exceeds 35 pixels. Dots are located from the recorded impressions with the die geometry. Success also requires every stamp released, still and supported. An untouched scene scores 0.

Odd Parcel. $P = \max(0, |B \cap O| / |B \cup O| - 0.1 [\text{balance moved}])$, where O is the set of odd parcels and B the set of parcels released in the answer box. An empty box scores 0, and parcels left on the pans are not penalized. Moving the balance more than 2 cm fails the episode.

Puzzle Box. The bolts of the chain and the item share P equally. Each bolt that is released at the end earns its share, and decoy bolts earn nothing. The item earns a third of its share once the lid is past half its travel, two thirds once the item has been out of the box, and its full share once it rests in the tray, released. Once the item has been out, every bolt counts as released. Success also requires that the robot touches no part at the press.

Wobbly Stand. $P = q(1 + b)/2$, where $b = 1$ if the ball is on top of the stand and 0 otherwise. With θ_0 and θ the initial and final tilt, $q = \text{clip}(1 - \theta/\theta_0, 0, 1)$ while the stand is released, settled and untouched, and $q = 1$ within the level threshold. $q = 0$ while the robot touches the stand, and values below 0.01 read 0. A level stand without the ball scores 0.5, and an untouched scene scores 0.

Table 10: **How episodes end**, in % of 500 episodes per model. The first five columns sum to 100. Only a correct submission succeeds. Precision is the share of submissions that were correct.

Model	Correct submission	Wrong submission	Stopped	Reply with no action	Out of decisions	Precision
GPT-6 Astra	23.2	61.8	5.4	0.0	9.6	27.3
Claude Opus 5.5	13.8	31.6	48.6	4.8	1.2	30.4
GPT-6.1 Sol	12.2	62.8	3.4	0.0	21.6	16.3
Claude Fable 5.1	11.4	53.6	28.6	5.4	1.0	17.5
Gemini 3.8 Flash	2.0	55.6	0.8	0.0	41.6	3.5

G EXECUTION SKILL TEST DETAILS

Every skill-test scene keeps the kitchen of an evaluation instance and holds only one object and its target from that task, with no SUBMIT button. The simulator checks the test’s goal after every physics tick and ends the episode once the goal has held for 0.5 s. Otherwise the episode ends after 50 decisions. Success is scored from the recorded episode by the source task’s own rule for that stage. The scenes vary what the skill depends on, such as straight and detour routes for driving or the eleven opening mechanisms of the containers. Every model plays the same 20 scenes per skill.

Skill selection. Every task ends by placing objects on or into a target on the counter, such as a tray or a bin. Most tasks also require driving the base to an object, the Search tasks most of all. Pick-and-place and driving are therefore the two general skills. The drawer and cabinet tests use the storage layout of the Search tasks. Marked Mugs, Odd Parcel, Stamp Composition and Puzzle Box mostly involve moving parts on the counter, which the general skills cover, so they have no test of their own.

Reaching the target. Table 11 compares how often the robot reached the target of a skill with how often it completed the skill.

Table 11: **Reaching the target and completing the skill**, in % of the 20 episodes per skill, GPT-6 Astra / Opus 5.5 / GPT-6.1 Sol.

Skill	Reached when	Reached	Completed
Open drawer/cabinet	the gripper touched the handle	85 / 85 / 75	50 / 60 / 30
Pick from drawer/cabinet	the gripper touched the object	90 / 80 / 100	70 / 60 / 60
Shim the short leg	the shim came within 3 cm of the leg	100 / 90 / 95	35 / 60 / 40
Drive and pick	the base came within 0.8 m of the object	100 / 100 / 95	100 / 100 / 90

Skills in the full tasks. Table 12 gives the two execution stages of retrieving a hidden target in the full tasks, beside the isolated test of the same skill (§5.2). A compartment counts as tried when the robot has touched or when the gripper closed within 12,cm of its handle. A grasp counts as tried when the robot has touched the target, or when the gripper closed within 12,cm of it. Decisions per opening are averaged over successful openings, from the last decision with the robot base at

least 0.2 m from the compartment’s standing position, where the isolated test starts. Decisions are compared only for opening, because there the two settings start alike. Taking a target out in the full tasks starts from however far the model opened the compartment and includes finding the target. Placing a shim in the full tasks includes finding the short leg and the right shim, while the isolated test names the leg and provides a single shim. For boxes, the isolated test names the opening mechanism, while in the full task the model must find it by trying.

Table 12: **Skills in the full tasks.** Opening the compartment or box of a hidden target once it was tried, and taking the target out once a grasp was tried. The last column gives the isolated test of the same skill (success %, GPT-6 Astra / Opus 5.5 / GPT-6.1 Sol).

Stage	GPT-6 Astra	Opus 5.5	GPT-6.1 Sol	Isolated test
<i>Search Room and Locked Storage: target in a compartment</i>				
Opened tried	52/58 (90%)	28/36 (78%)	24/32 (75%)	Open drawer/cabinet: 50 / 60 / 30
Decisions per opening (mean)	10.4	25.7	15.2	Open drawer/cabinet: 22.1 / 30.8 / 35.8
Taken out grasp tried	30/38 (79%)	17/18 (94%)	13/16 (81%)	Pick from drawer/cabinet: 70 / 60 / 60
<i>Unfamiliar Containers: item in a box</i>				
Opened tried	57/81 (70%)	35/77 (45%)	49/63 (78%)	Open named container: 100 / 90 / 95
Taken out grasp tried	43/44 (98%)	24/26 (92%)	28/31 (90%)	Pick from open container: 90 / 90 / 70

H DEMONSTRATION DATASET DETAILS

Table 13 lists the scripted information gathering of each oracle and the size of each task’s dataset. The rest of this appendix gives the language annotations and the release format.

Language annotations. Every frame stores the three camera views, the robot state, the action, and two levels of language annotation. A *stage* such as “Open and look inside the nearer of the two boxes on your right” contains finer *subtasks* such as “Try pulling the knob on the front of the box”. Objects are named only by their appearance and position as seen from the robot at that moment. Hidden information is never named before the robot has seen it. For example, an item in a closed box is not named until the box is open. Every control step belongs to exactly one subtask. Stages last 10 to 50 s and subtasks about 10 s, and Table 13 gives the mean number of subtasks per task.

Format. The dataset is released in the LeRobot v2.1 format, one dataset per task. Each frame holds the three camera views as 256×256 video at 20 Hz, the 16-dimensional robot state and the 12-dimensional action. The goal text is the task string, and per-frame indices point into the stage and subtask tables. The release scripts regenerate the datasets at other resolutions from the recorded states.

Table 13: **Demonstration dataset.** Scripted information gathering in each oracle’s route, which the oracle performs although it knows the hidden information. Every task has 500 successful, verified episodes. Minutes: mean episode length. Hours: interaction time at 20 Hz. Subtasks: mean number of subtask annotations per episode.

Task	Scripted information gathering	Minutes	Hours	Subtasks
Locked Storage	follows the token chain	3.7	31.1	16
Search Room	opens compartments nearest-first until the targets are seen	3.0	25.1	11
Blackout Search	carries the lamp along a nearest-first search	7.5	62.1	33
Painted Cubes	removes covers and turns each cube face by face	5.1	42.4	14
Marked Mugs	lifts and tilts each vessel to read its label	3.4	28.0	15
Unfamiliar Containers	visits boxes nearest-first and tries each knob’s actions until one opens the box	6.4	53.0	29
Puzzle Box	also pulls blocked bolts on the way	1.6	13.7	10
Stamp Composition	prints test impressions before the final one	3.6	30.1	22
Wobbly Stand	releases the ball, places a shim, and tests again	2.8	23.2	11
Odd Parcel	weighs parcels on the balance	6.9	57.8	42
Total (10 × 500 episodes)		4.4	366.4	20

I WORKLOAD AND OCCLUSION DETAILS

This section reports how the two difficulty parameters affect the three models, and §5.2 uses the occlusion result. Painted Cubes, Marked Mugs, Stamp Composition and Odd Parcel vary occlusion at three levels, and Unfamiliar Containers at the first two. In *Visible*, every task object is in a camera view at the start. In *Look*, some object starts out of view, for example behind a tall object, and the robot must move to see it. In *Uncover*, some object is under a cover that the robot must remove, for example a cloche to lift or a board to push aside. Workload is the number of units a task requires, for example the number of cubes or the depth of the lock chain, and six tasks vary it at three levels. Table 14 gives success and progress per level and Table 15 the failure families per level. Figure 7 gives the Painted Cubes error rates by rule.

Table 14: **Occlusion and workload.** Success and progress (%) by level, pooled over the three models and per model. Occlusion on the four tasks that vary it, workload on the six.

	SR				Prog			
	GPT-6 Astra	Opus 5.5	GPT-6.1 Sol	Pooled	GPT-6 Astra	Opus 5.5	GPT-6.1 Sol	Pooled
<i>Occlusion (4 tasks)</i>								
Visible	28.8	16.3	14.0	19.7	57.6	45.8	39.5	47.7
Look	16.3	7.8	5.0	9.7	53.5	38.6	32.6	41.6
Uncover	22.3	7.5	6.1	12.0	53.1	35.9	34.1	41.1
Look – Visible	−12.5	−8.6	−9.0	−10.0	−4.1	−7.2	−7.0	−6.1
Uncover – Visible	−6.5	−8.8	−7.9	−7.7	−4.6	−9.9	−5.4	−6.6
<i>Workload (6 tasks)</i>								
Low	35.6	16.1	17.2	23.0	58.0	43.6	39.2	47.0
Mid	15.4	4.0	4.6	8.0	50.0	30.2	33.3	37.8
High	10.5	4.1	1.1	5.2	43.1	32.9	29.0	35.0
High – Low	−25.1	−12.1	−16.1	−17.7	−14.9	−10.8	−10.2	−12.0

J FAILURE ATTRIBUTION DETAILS

Requirements. We attribute failures per requirement rather than per episode, as a single episode can fail in multiple ways. An episode’s requirements contains both its goal objectives (such as

Table 15: **Where the extra failures come from.** Unmet requirements per episode by failure family (execution failures and side effects summed), pooled over GPT-6 Astra, Opus 5.5 and GPT-6.1 Sol. Occlusion on the four tasks that vary it, workload on the six.

	Missing	Wrong	Execution + side effect	All
<i>Occlusion (4 tasks)</i>				
Visible	0.62	1.24	0.72	2.58
Look	0.89	1.32	0.53	2.74
Uncover	0.94	1.28	0.72	2.94
Look – Visible	0.27	0.08	−0.18	0.16
Uncover – Visible	0.32	0.04	0.00	0.36
<i>Workload (6 tasks)</i>				
Low	0.53	0.73	0.43	1.69
Mid	0.96	1.12	0.64	2.73
High	1.59	1.12	0.68	3.39
High – Low	1.06	0.39	0.25	1.70

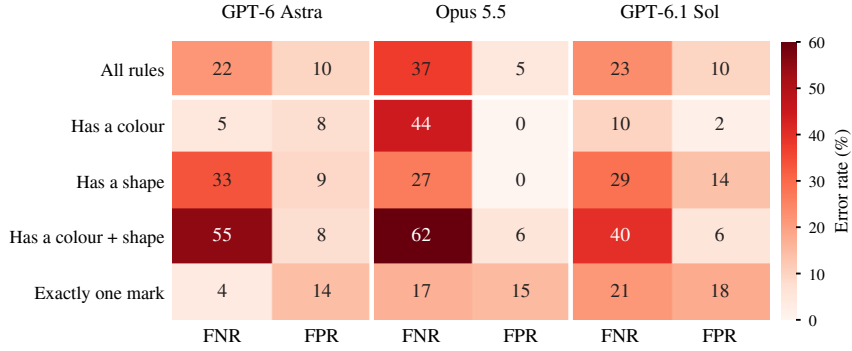


Figure 7: **Painted Cubes: error rates by rule.** False-negative rate (FNR) and false-positive rate (FPR) over committed cubes, in %.

each target that must reach the tray) and its task constraints (such as keeping other objects off the tray). Each unmet requirement serves as an individual unit of analysis in our attribution procedure (Algorithm 1).

Units. Each unmet requirement in the final simulator state of an episode constitutes one unit, such as a target left off the tray or an incorrect stamp cell. When a single error leaves multiple requirements unmet, each requirement is treated as a distinct unit and attributed independently. Across all tasks, this yields 931 units for GPT-6 Astra from 384 failed episodes, 1,143 for Opus 5.5 from 431, and 1,182 for GPT-6.1 Sol from 439.

Weighting. Each failed episode carries equal weight, split evenly across its units. We compute each family’s share as the weighted fraction of its units. We also tried another aggregation scheme by pooling all units and averaging the ten per-task shares. Under both schemes, missing evidence remains the largest family for every model. Missing evidence and wrong decisions together account for 66–78% of failures, and no family share shifts by more than 7 percentage points (Table 16).

Measurements. Attribution relies on data logged during execution: the simulator state recorded at every 20 Hz tick and the three camera views delivered to the model at each decision step. Replaying these simulator states reproduces the exact scene geometry, allowing us to render ground-truth segmentation masks for each delivered observation. These masks identify the visible pixels of each object and key features (e.g., cube markings or container labels), whose pixel brightness is measured directly from the delivered images. From the physical trajectory, we also extract interaction events, including grasps, drops, final resting poses, opened compartments, and stamped ink impressions.

Algorithm 1 Failure attribution over the failed episodes of one agent.

Require: For each failed episode e : the private task spec, the simulator state at every 20 Hz tick, the commands, and the images $I_1^e, \dots, I_{T_e}^e$ delivered to the model at its decision ticks

Ensure: Share of failures in each family

```

1:  $w[\cdot] \leftarrow 0$  ▷ weight per family
2: for all failed episodes  $e$  do
3:    $U_e \leftarrow$  unmet requirements of  $e$  in its final state, as defined per task
4:   for all units  $r \in U_e$  do ▷ every unmet requirement, not only the first
5:      $w[\text{LABEL}(r)] \leftarrow w[\text{LABEL}(r)] + 1/|U_e|$  ▷ each failed episode has total weight 1
6:   end for
7: end for
8: return  $w[\ell] / \sum_{\ell'} w[\ell']$  for each family  $\ell$ 

9: function LABEL( $r$ )
10:  Follow the events that led to  $r$ 's final state, ignoring errors that were later repaired
11:  for all rules  $\rho$  of  $r$ 's task, in their fixed order do
12:    if  $\rho$  applies to  $r$  then return the family of  $\rho$ 
13:  end if
14: end for
15: end function

16: function SEEN( $f, t$ ) ▷ called by rules that ask whether the agent saw  $f$ 
17:  for all decision ticks  $t' < t$  do
18:     $P \leftarrow$  pixels of  $f$  in  $I_{t'}^e$ , from a segmentation rendered from the state at  $t'$ 
19:     $b \leftarrow$  mean brightness of  $P$ , read from the saved image  $I_{t'}^e$ 
20:    if  $|P|$  and  $b$  reach the thresholds for  $f$  then return true
21:  end if
22: end for
23: return false
24: end function

```

Decision procedure. Algorithm 1 outlines the decision procedure. For each unit, we trace the sequence of events and identify the first error that was not subsequently repaired. Each task defines an ordered list of deterministic rules. We evaluate these rules sequentially to assign the failure family to each unit based on the first rule that matches. A failed recovery attempt is classified as an execution failure, whereas an unnoticed error is attributed to a wrong decision if the deciding evidence was observed by the agent, and to missing evidence if it was not. The entire procedure is programmatic and deterministic, evaluated directly on recorded run logs.

Family boundaries. Several criteria are used to resolve edge cases between failure families. For premature terminations (stopping or submitting early), the unit is attributed to missing evidence if the agent stopped before observing the deciding evidence, and to a wrong decision if it stopped after observing the evidence. A failed physical attempt to uncover evidence (such as opening a drawer insufficiently) is attributed to execution failure rather than missing evidence. Finally, exploratory actions that perturb the scene (such as tipping a mug or rolling a test ball) are valid, so the unit is classified under side effect only to damage left unrepaired. Whether such damage was an unforeseen consequence or the result of clumsy execution cannot be determined from the recorded data.

Thresholds. To determine whether an object was observed by the agent (SEEN in Algorithm 1), we evaluate its visibility across the delivered camera views. The saved simulator state identifies which pixels belong to the object, and the delivered image gives their brightness. An object counts as observed when its pixel count and mean brightness both meet fixed thresholds, set to ≥ 150 visible pixels in one delivered image with mean brightness ≥ 25 (≥ 3 in Blackout Search). In Odd Parcel, a parcel counts as determined when the balance readings the agent saw are enough to determine whether the parcel is odd or normal. The remaining thresholds, the rules of every task and the per-unit labels are released with the benchmark.

Table 16: **Failure attribution per task**, in units (unmet requirements of failed episodes). Ran out: units in the Missing or Wrong columns whose episode ended by the decision budget.

Task	Policy	Failed episodes	Units	Missing	Wrong	Side effect	Execution	Ran out
Locked Storage	GPT-6 Astra	36	87	48	23	0	16	7
	Opus 5.5	47	118	76	29	0	13	1
	GPT-6.1 Sol	44	108	69	30	0	9	42
Search Room	GPT-6 Astra	50	121	67	25	0	29	32
	Opus 5.5	50	118	81	19	0	18	0
	GPT-6.1 Sol	50	132	87	18	0	27	59
Blackout Search	GPT-6 Astra	50	128	73	14	0	41	9
	Opus 5.5	50	122	83	14	0	25	0
	GPT-6.1 Sol	50	127	81	16	2	28	18
Painted Cubes	GPT-6 Astra	35	80	38	34	1	7	0
	Opus 5.5	43	124	46	67	0	11	4
	GPT-6.1 Sol	41	96	40	44	0	12	4
Marked Mugs	GPT-6 Astra	31	80	5	24	19	32	0
	Opus 5.5	42	113	7	55	11	40	0
	GPT-6.1 Sol	47	149	5	48	24	72	7
Unfamiliar Containers	GPT-6 Astra	47	111	70	13	0	28	15
	Opus 5.5	50	126	79	11	0	36	0
	GPT-6.1 Sol	50	125	83	15	0	27	38
Puzzle Box	GPT-6 Astra	2	2	1	0	0	1	0
	Opus 5.5	16	26	1	22	0	3	0
	GPT-6.1 Sol	14	17	3	10	0	4	3
Stamp Composition	GPT-6 Astra	43	195	45	102	17	31	0
	Opus 5.5	49	260	9	193	15	43	0
	GPT-6.1 Sol	46	280	85	167	16	12	4
Wobbly Stand	GPT-6 Astra	45	45	2	6	36	1	1
	Opus 5.5	38	38	4	11	23	0	0
	GPT-6.1 Sol	48	48	3	4	40	1	2
Odd Parcel	GPT-6 Astra	45	82	60	9	4	9	10
	Opus 5.5	46	98	70	12	1	15	6
	GPT-6.1 Sol	49	100	84	10	0	6	16
All tasks	GPT-6 Astra	384	931	409	250	77	195	74
	Opus 5.5	431	1143	456	433	50	204	11
	GPT-6.1 Sol	439	1182	540	362	82	198	193

Breakdown by physical outcome. Table 17 separates failures in the two failure families (Missing evidence and Wrong decision) by the agent’s physical action. An object is considered as never placed if it remains in an unopened compartment or is put down short of its destination. An object is considered as placed wrongly if it is committed before deciding evidence is uncovered, or placed in contradiction to observed evidence.

Table 17: **Missing evidence and wrong decisions by what the agent did**, in % of failures, each failed episode weighted equally.

Family	Object	GPT-6 Astra	Opus 5.5	GPT-6.1 Sol
Missing evidence	Never placed	31.9	33.5	34.2
	Placed before the evidence decided	11.2	9.4	11.4
Wrong decision	Never placed	12.8	19.1	15.9
	Placed against the evidence	10.2	11.9	9.3
Both	Never placed	44.6	52.6	50.1
	Placed wrongly	21.4	21.3	20.7